

Session Report

Spatial Graph & Collision System Updates

AIA26 Studio - Team 03

Industrial Spatial Flow Agent

Date: May 22, 2026

Files modified: collision.py, spatial_graph.py

Files updated: MASTER_CLAUDE_V2.md

1. Session Summary

Two improvements were made to the spatial analysis pipeline on this session:

1. Fixed the collision clearance computation (`collision.py`) so that `min_clearance_m` reports the real gap between each object and its nearest neighbor, instead of a constant 0.1m (1 grid cell) for every object.
2. Added walls (structure) and windows as first-class nodes in the spatial graph (`spatial_graph.py`), with `near_wall` and `near_window` proximity edges to furniture.
3. Fixed `clearance_ok` evaluation in `spatial_graph.py` to use `deficit_m > 0` instead of just checking for the presence of a `clearance_violation` dict.

2. Collision Clearance Fix (Voronoi Boundary Method)

The Problem

All 19 furniture/MEP objects in the test layout showed identical clearance values: 'has 0.1m, needs 0.4m' with a deficit of -0.3m. This happened because `min_clearance_m` was computed inside the violation cell loop, tracking the minimum `dist * cell_size` for cells attributed to each object. Since `dist` measures BFS distance from any free cell to the nearest obstacle surface, and the cells immediately adjacent to any object always have `dist=1` (= 0.1m at 0.10m grid resolution), every object bottomed out at 0.1m.

```
BEFORE (all identical):
CLEARANCE Toilet: has 0.1m, needs 0.4m -> move [...] 0.4m
CLEARANCE Sink: has 0.1m, needs 0.4m -> move [...] 0.4m
CLEARANCE Assembly Station 1: has 0.1m, needs 0.4m -> move [...] 0.4m
CLEARANCE cnc_machine: has 0.1m, needs 0.4m -> move [...] 0.4m
... (all 19 objects show 0.1m)
```

The Solution

After the violation cell loop, a Voronoi boundary scan computes the real gap between each object and its nearest other obstacle (wall, furniture, or MEP). The algorithm:

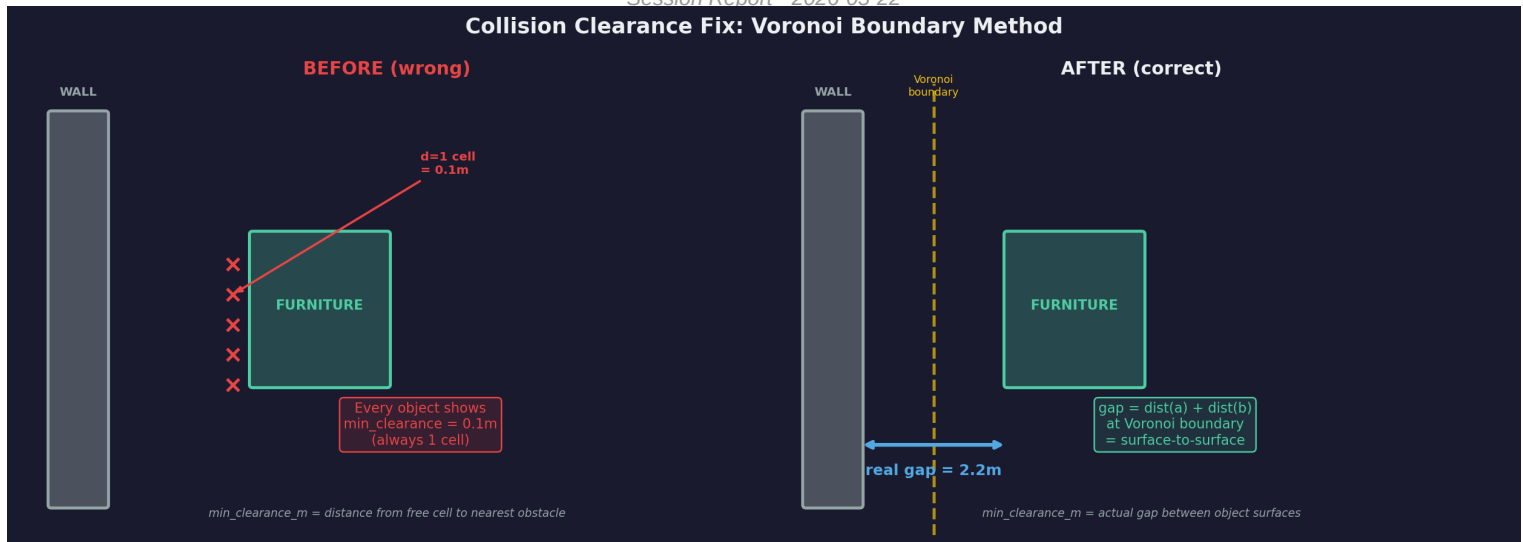
1. Iterate all free cells in the grid (`dist > 0`)
2. For each free cell, check its 4-connected neighbors
3. If a neighbor's nearest-obstacle attribution differs (`nearest[cell] != nearest[neighbor]`), we are at a Voronoi boundary between two obstacles
4. The gap at that boundary = $(\text{dist}[\text{cell}] + \text{dist}[\text{neighbor}]) * \text{cell_size}$
5. Track the minimum gap for each object across all its Voronoi boundaries

This gives the actual surface-to-surface distance between the object and its nearest neighbor, not the per-cell distance to any obstacle.

```
# Core of the Voronoi boundary scan:
for idx in range(total):
    d = dist[idx]
    if d <= 0: continue
    oi = nearest[idx]
    for neighbor in 4_connected(idx):
        dn = dist[neighbor]
        oj = nearest[neighbor]
        if oj != oi: # Voronoi boundary!
            gap = (d + dn) * cell_size
            obj_real_clearance[oi] = min(gap, ...)
            obj_real_clearance[oj] = min(gap, ...)
```

```
AFTER (differentiated per object):
CLEARANCE Toilet: has 0.35m, needs 0.4m -> move [...] 0.15m
Assembly Station 1: clearance=OK (gap 1.2m > 0.4m required)
cnc_machine: has 0.28m, needs 0.4m -> move [...] 0.22m
```

Collision Clearance Fix: Voronoi Boundary Method



No changes were needed in the Grasshopper component. The GH script independently re-runs the grid analysis for visualization only. The Python side (collision.py) is the authoritative source for all clearance values consumed by the agent and spatial graph.

3. Walls and Windows in the Spatial Graph

What was missing

The spatial graph previously included 4 element types: rooms, doors, furniture, and MEP. The layout JSON has 7 layers. Of the missing 3 (outline, structure, windows), structure and windows carry important spatial information:

- Structure (walls): 6 walls in the test layout (4 exterior load-bearing, 2 interior partitions). Essential for knowing if furniture is against a wall, which walls are structural vs. partition, and wall-proximity relationships.

What was added

New node types:

```
wall node:
  ntype='wall', name, wall_type (load-bearing/partition),
  material (concrete/drywall), length, p1, p2 (segment endpoints)

window node:
  ntype='window', name, window_type (awning/sliding/fixed),
  roomId, width, p1, p2 (segment endpoints)
+ contained_in edge to parent room
```

New edge types:

```
near_wall:
  furniture -> wall, distance_m (point-to-segment), threshold 3m
  Uses _point_to_segment_distance() for accurate projection

near_window:
  furniture -> window, same room only, same distance method
```

New helper function:

```
def _point_to_segment_distance(px, py, x1, y1, x2, y2):
    # Orthogonal projection of point onto segment,
    # clamped to endpoints. Returns shortest distance.
    t = clamp(0, 1, dot(P-A, B-A) / |B-A|^2)
    return dist(P, A + t*(B-A))
```

Serialization changes:

- STRUCTURE: section lists each wall with type and length

5. Updated LLM Serialization

The `serialize_for_llm()` output now includes **STRUCTURE** and **WINDOWS** sections. Below is the complete output from `industrial_005` (base layout, no analysis enrichment):

```
SPATIAL GRAPH (45 nodes, 43 edges)

ROOMS:
  room-1 "Bathroom" area=45.5m2
  room-2 "Clean Room" area=812.5m2
CONNECTIVITY:
  Bathroom <--door-1(0.9m)--> Clean Room
STRUCTURE:
  wall-1 "South Exterior Wall" load-bearing 52.0m
  wall-2 "East Exterior Wall" load-bearing 16.5m
  wall-3 "North Exterior Wall" load-bearing 52.0m
  wall-4 "West Exterior Wall" load-bearing 16.5m
  wall-5 "Interior Wall 5" partition 6.5m
  wall-6 "Interior Wall 6" partition 7.0m
WINDOWS: Clean Room: 16 (5x awning, 8x sliding, 3x fixed)
FURNITURE in Bathroom:
  furn-1 "Toilet" at(45.5,10.6)
  furn-2 "Sink" at(46.3,10.5)
FURNITURE in Clean Room:
  furn-3 "Assembly Station 1" at(7.1,10.5)
  furn-4 "Packaging Station 2" at(31.9,10.6)
  furn-5 "Packaging Station 3" at(14.3,5.5)
  furn-6 "Packaging Station 4" at(6.7,7.4)
  furn-7 "Conveyor Section 5" at(29.7,4.5)
  furn-8 "Labeling Station 6" at(25.8,3.1)
  furn-9 "Parts Bin Rack 7" at(9.0,5.3)
  furn-10 "Conveyor Section 8" at(18.8,12.8)
  furn-11 "Parts Bin Rack 9" at(19.0,2.5)
  furn-12 "Conveyor Section 10" at(39.0,12.3)
  furn-13 "Parts Bin Rack 11" at(28.5,9.5)
  furn-14 "QC Table 12" at(37.1,10.3)
MEP:
  mep-1 "HVAC Unit 1" system=hvac
  mep-2 "HVAC Unit 2" system=hvac
  mep-3 "Main Electrical Panel" system=electrical
  mep-4 "Plumbing Riser" system=plumbing
RELATIONS:
  Toilet --near(0.88m)--> Sink
  Toilet --near_wall(0.46m)--> Interior Wall 5
  Toilet --near_wall(0.56m)--> Interior Wall 6
  Sink --near_wall(1.34m)--> Interior Wall 5
  Sink --near_wall(0.48m)--> Interior Wall 6
  Parts Bin Rack 9 --near_wall(2.5m)--> South Exterior Wall
  Parts Bin Rack 9 --near_window(2.7m)--> Warehouse Window 4
  Conveyor Section 10 --near(2.82m)--> QC Table 12
```

6. clearance_ok Evaluation Fix

In `spatial_graph.py`, the `enrich_graph_from_analysis()` function previously set `clearance_ok = False` whenever a `clearance_violation` dict existed for an object, regardless of the deficit value. With the Voronoi fix, some objects may have a `clearance_violation` entry (for heatmap data) but with `deficit_m = 0` (sufficient gap).

Fix: `clearance_ok` is now based on `deficit_m <= 0`:

```
# BEFORE:
if cv:
    G.nodes[oid]['clearance_ok'] = False # always False if cv exists

# AFTER:
if cv:
    deficit = cv.get('deficit_m', 0)
    G.nodes[oid]['clearance_ok'] = deficit <= 0 # OK if no real deficit
```


7. Files Modified

nodes/collision.py -- Voronoi boundary scan

Added ~35 lines after the violation cell loop. Scans all free cells for Voronoi boundaries (adjacent cells with different nearest-obstacle attribution). Computes real surface-to-surface gap. Overwrites min_clearance_m in obj_violation_data. Objects with no boundary (touching another obstacle) get min_clearance_m = 0.0.

spatial_graph.py -- Walls, windows, clearance_ok fix

Added _point_to_segment_distance() helper. Added wall and window node construction in build_graph_from_layout(). Added near_wall and near_window edge computation. Added STRUCTURE and WINDOWS sections to serialize_for_llm(). Fixed clearance_ok to use deficit_m <= 0. Increased MAX_SERIALIZE_LINES to 80.

MASTER_CLAUDE_V2.md -- Documentation update

Updated spatial graph node/edge tables, collision.py description, spatial_graph.py description, terminal output example, and added changelog section for 2026-05-22.